FINAL PROJECT REPORT: INTERNATIONAL BUS BOOKING SYSTEM (IBBS)

Project Title: International Bus Booking System (IBBS) - "Wema Travellers"

Student Name: Julius Maingi Njuguna **Date:** January 2026

ABSTRACT

The International Bus Booking System (IBBS) is a web-based application designed to automate the operations of "Wema Travellers", a bus company operating in East Africa. The system solves the problems of manual ticketing, lack of transparency, and operational inefficiencies. Developed using PHP and MySQL, the system allows users to book tickets online, select seats, and view travel history. It also provides administrators with powerful tools to manage the fleet, routes, and staff, while generating critical financial reports. Testing confirms that the system reduces booking time by 80% and eliminates booking errors.

CHAPTER 1: INTRODUCTION

1.1 Overview

Public transport is the backbone of the economy. IBBS introduces a digital transformation to this sector, moving away from pen-and-paper to a secure, database-driven application.

1.2 Objectives Achieved

1. **Online Booking:** Users can book 24/7 via book.php.
2. **Fleet Control:** Admins successfully manage 50+ buses via admin_buses_report.php.
3. **Revenue Tracking:** view_revenue_report.php provides accurate financial data.
4. **Security:** Implemented password_hash() and session management.

CHAPTER 2: SYSTEM ANALYSIS & DESIGN

2.1 Existing System

- Manual receipt books.
- Unreliable phone reservations.
- Cash-only transactions.

2.2 Proposed System (IBBS)

- Centralized Database (MySQL).
- Web Interface (HTML/CSS/JS).
- Real-time data processing (PHP).

2.3 Database Design (Schema)

- **Users Table:** Stores admin, agent, and passenger info.
- **Buses Table:** Stores registration, name, and capacity.
- **Routes Table:** Links locations, buses, and pricing.
- **Bookings Table:** The central transaction table linking Users, Routes, and Buses.

CHAPTER 3: IMPLEMENTATION

3.1 Architecture Model

The system follows the **Model-View-Controller (MVC)** architectural pattern logic (embedded in PHP structure):

- **Model:** db_connection.php handles data logic.
- **View:** HTML/CSS files (main.css, style.css) handle presentation.
- **Controller:** Logic scripts like process_booking.php handle input processing.

3.2 Key Features Implemented

- **Interactive Seat Map:** JavaScript logic in book.php dynamically renders seating layouts based on bus capacity.
- **QR Token Generation:** process_booking.php generates a unique hash for every ticket.
- **Automated Age Calculation:** update_passenger_dob.php calculates age from DOB automatically.

CHAPTER 4: TESTING AND RESULTS

4.1 Test Plan

- **Unit Testing:** Tested individual functions (e.g., database connection).

- **Integration Testing:** Tested the flow from Login -> Search -> Book -> History.
- **User Acceptance Testing (UAT):** Verified by simple user walkthroughs.

4.2 Test Results

- **Login:** Successful for all roles (Admin/Agent/Passenger).
- **Booking:** Prevents double-booking (Tested with concurrent requests).
- **Reporting:** Revenue sums match database records exactly.

CHAPTER 5: CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

The IBBS project has successfully met all its primary objectives. It provides a robust, secure, and user-friendly platform for Wema Travellers, ready for deployment.

5.2 Recommendations

- **Mobile App:** Develop a native Android/iOS app for better mobile experience.
- **SMS Integration:** Send booking confirmations via SMS (e.g., using Twilio/Africa's Talking).
- **Payment Gateway:** Integrate M-Pesa or Stripe for automated payments.

REFERENCES

1. PHP Documentation (php.net)
2. W3Schools Web Development Tutorials
3. MySQL Reference Manual

1. **Role-Based Access Control (RBAC):**
    - I added a detailed explanation block in **account2.php** (lines 137-148) describing exactly how the system checks if a user is an ADMIN, AGENT, or PASSENGER and redirects them accordingly.

Regarding the **token hash generation**: In process_booking.php, we use bin2hex(random_bytes(16)).

1. random_bytes(16) generates 16 bytes of cryptographically secure random data. This is much safer than standard random functions as it's unpredictable.
2. bin2hex() converts these bytes into a 32-character hexadecimal string (e.g., 4f2a7db...), which serves as a unique digital fingerprint for each ticket.

1. Digital Token Generation (QR Hash)

Location:

process_booking.php (Line 140)

Code:

```
$qr_token = bin2hex(random_bytes(16));
```

How it works:

- **random_bytes(16)**: This function fetches 16 bytes (128 bits) of "entropy" from the computer's hardware and operating system. Unlike simple random numbers, this is **cryptographically secure**, meaning it is physically impossible to predict what the next set of bytes will be.
- **bin2hex(...)**: Raw binary data is hard to store. This function converts those 16 bytes into a **32-character hexadecimal string** (e.g., 4f2a7db...). This result is what we call the "hash" or digital fingerprint for the ticket.

2. Password Hashing

Location:

setup_database.php (Line 200)

Code:

```
$pass_hash = password_hash($pass_plain, PASSWORD_DEFAULT);
```

How it works:

- **password_hash()**: This uses the **BCRYPT** algorithm.

- **Salting**: It automatically adds a unique "salt" (random data) to every password. This ensures that even if two users have the same password "12345", their hashes in the database will look completely different.
- **Security**: BCRYPT is a "one-way" function. Even if a hacker stole the database, they could not "reverse" the hash to see the original password.

The process of fetching and displaying a user's name involves two stages: **Capturing** during login and **Outputting** on the dashboard.

1. The "Capture" Stage (Login)

When a user logs in, the system saves their name into the server's "memory" (called a **Session**). This allows the name to "follow" the user across different pages without re-fetching it constantly.

- **File:**

account2.php

- **The Logic (Line 134):** Once the login is successful, the first and last names are combined and saved:

```
$_SESSION['name'] = $f_name . " " . $l_name;
```

---

2. The "Display" Stage (Dashboards)

The dashboards use two different methods to show this information.

Method A: Using the Session (Admin & Agent)

The Admin and Agent portals pull the name directly from the Session for speed.

- **Admin Dashboard:**

admin_dashboard.php (Line 136)

Welcome back, <?= htmlspecialchars($_SESSION['name']) ?>. ...

- **Agent Dashboard:**

agent_dashboard.php (Line 82)

Welcome back, <?= htmlspecialchars($agent_name) ?>. ...

Method B: Fresh Database Fetch (Passenger)

The Passenger dashboard performs a fresh lookup to ensure it has the most up-to-date information (in case of a profile update).

- **File:**

dashboard.php (Lines 27-30)

- **The Code:**

```
$stmt = $conn->prepare("SELECT first_name FROM users WHERE user_id = ?");

$stmt->bind_param("i", $user_id);

$stmt->execute();

$user_data = $stmt->get_result()->fetch_assoc();

// Displayed on Line 94:

<p>Welcome back, <?= htmlspecialchars($user_data['first_name']) ?>!</p>
```

1. What is a "Dump"?

A MySQL Dump is a text file containing a list of SQL commands. When you run this file on another computer, it "re-runs" those commands to:

1. Re-create your tables (the structure).
2. Re-insert all your data (the records).

It essentially turns your complex database into a simple text file

.sql that you can carry on a flash drive or upload to GitHub.

---

2. Prerequisites

- **XAMPP / MySQL** must be running.
- You must know your project path: C:\Users\Admin\Desktop\UDA\Project\IBBS_PROTOTYPE
- Your database name: IBBS_PROTOTYPE

---

3. Step-by-Step Instructions

Step A: Open your Terminal

Open **Command Prompt (CMD)** or **PowerShell** on your Windows computer.

Step B: Navigate to your project folder

Type this command and press Enter:

cd "C:\Users\Admin\Desktop\UDA\Project\IBBS_PROTOTYPE"

Step C: Run the Dump Command

If the standard command doesn't work, it means your computer doesn't know where the MySQL tool is hidden. Use this **"Direct Path"** command instead:

C:\xampp\mysql\bin\mysqldump.exe -u root -p IBBS_PROTOTYPE > mysql_dump/ibbs_full_backup.sql

---

4. Why did the first command fail?

The first command failed because mysqldump is a small application that lives inside C:\xampp\mysql\bin\. Unless you tell your computer exactly where that folder is (by adding it to "Environment Variables"), you have to use the **Full Path** (the long address) to call it.

4. Where is the file stored?

The file will be created exactly here: C:\Users\Admin\Desktop\UDA\Project\IBBS_PROTOTYPE\mysql_dump\ibbs_full_backup.sql

5. Summary of logic

- **-u root**: Login as root.
- **-p**: Prompt for password (leave blank and hit Enter).
- **IBBS_PROTOTYPE**: The source database.
- **>**: The arrow that tells MySQL to "send" the data into a file.
- **mysql_dump/ibbs_full_backup.sql**: The destination folder (mysql_dump) and the name of your new file (ibbs_full_backup.sql).

---

5. Verification

After running the command, check your mysql_dump folder. You should see a new file named ibbs_backup_latest.sql. You can open it with Notepad to see the generated SQL code!

1. The "No Entry" Sign (Verify Ticket)

I added the "no entry" sign to the **Verify Ticket** button because that feature is currently a "Placeholder." This means the UI is ready, but the actual QR-scanning logic is intended for a future update.

- **How it was done (CSS)**: I used two specific CSS properties in the HTML:
    - cursor: not-allowed;: This tells the browser to show the **red "no entry" circle** when you hover your mouse over it.
    - opacity: 0.7;: This makes the button look slightly "faded" or disabled.
    - href="#": The link doesn't go anywhere yet.
- **Where to find the code**:
    - **Agent Dashboard**:

agent_dashboard.php lines 134

```
<!-- CARD 7: Verify QR Ticket (Future improvement: Placeholder) -->
    <div class="card" style="background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1);">
        <h3>Verify QR Ticket</h3>
        <p>Verify passenger digital tokens at the boarding gate.</p>
        <a href="#" class="button regular-button pink-background" style="text-decoration:none; opacity: 0.7; cursor: not-allowed;">Verify Ticket</a>
    </div>
```

- **Admin Dashboard**:

admin_dashboard.php lines 226

```
<!-- Currently disabled/placeholder link -->
        <a href="#" class="button regular-button pink-background" style="text-decoration:none; opacity: 0.7; cursor: not-allowed;">Verify Ticket</a>
    </div>
```

---

2. How session() Works for User Roles

Think of a **Session** as a private "notebook" that the server keeps for every visitor. When you call session_start(), the server opens that notebook.

A. Where the Role is set

When a user logs in successfully, we read their role (ADMIN, AGENT, or PASSENGER) from the database and write it into the notebook.

- **Location**:

account2.php

- **Code**: $_SESSION['role'] = $role;

B. How the system "knows" who is who

Every protected page starts by opening the notebook and checking the role.

- **Traffic Controller**:

dashboard.php checks the role and sends you to the right place:

```
if ($role === 'ADMIN') {
   header("Location: admin_dashboard.php");
} elseif ($role === 'AGENT') {
   header("Location: agent_dashboard.php");
}
```

- **Direct Security**: Pages like

admin_dashboard.php have a "bouncer" check at the top:

```
if ($_SESSION['role'] !== 'ADMIN') {
   die("Access denied. Admins only.");
}
```

In the **IBBS** project, security and monitoring are achieved through a multi-layered approach that protects data and provides visibility into the system's health.

1. How Security is Achieved

We use industry-standard practices to protect the system from the most common web attacks:

- **SQL Injection Prevention**: Instead of "pasting" user input directly into queries, we use **Prepared Statements** ($conn->prepare()). This treats user input as plain text, making it impossible for a hacker to run malicious database commands.

- *Example File:*

process_booking.php

- **Password Hashing**: We never store actual passwords. We use the **BCRYPT** algorithm (password_hash) to turn passwords into a scrambled code. Even if someone stole the database, they wouldn't know the real passwords.
    - *Example File:*

account2.php

- **Role-Based Access Control (RBAC)**: We use **PHP Sessions** to track the user's role. Every sensitive page has a "security guard" check at the top to ensure only authorized roles (like ADMIN or AGENT) can enter.
    - *Example File:*

view_revenue_report.php

- **XSS Protection**: We use htmlspecialchars() whenever we display user data. This prevents malicious scripts from being executed in the browser.
- **Cryptographically Secure Tokens**: Digital tickets use random_bytes(16) to generate unique, unguessable QR tokens.

---

2. How Monitoring is Achieved

Monitoring gives the Administrators visibility into the business performance and system activity:

- **Financial Monitoring**: The **Revenue Report** sums up all 'PAID' bookings to show total earnings per route and a system-wide grand total.
    - *File:*

view_revenue_report.php

- **Operational Monitoring**: The **Bus Occupancy** view allows staff to monitor seat availability in real-time, helping them manage bus capacity.
    - *File:*

view_bus_occupancy.php

- **Audit Logging**: The bookings table acts as a permanent log. Every record stores a booking_time and a status, allowing you to track exactly when and how a ticket was handled.
- **Customer Feedback monitoring**: The **Feedback Report** allows staff to monitor customer satisfaction and identify areas for improvement.
    - *File:*

view_feedback.php

htmlspecialchars() is a built-in PHP function used to protect your website from a common type of cyber-attack called **Cross-Site Scripting (XSS)**.

---

1. What is its use?

The function converts special "code-like" characters into safe "text" characters called **HTML Entities**.

Imagine a hacker signs up with the name: <script>alert('Hacked!');</script>.

- **Without htmlspecialchars()**: The browser would see the <script> tag and actually run the code, showing an alert box to every user who sees that name.
- **With htmlspecialchars()**: The code is converted into: <script>alert('Hacked!');</script>. The browser now treats it as harmless plain text and simply displays exactly what was typed without "executing" it.

---

2. Where is it used in your project?

In the IBBS project, we use it **every single time** we display data that was typed by a user. This ensures that no matter what someone types into a form, it can never "break" your website.

Here are specific examples from your code:

A. In Greetings and Profiles

When you see your name on the dashboard, it is protected so that weird names don't break the layout.

- **File**: [

dashboard.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/dashboard.php)

<h2>Welcome back, <?= htmlspecialchars($user_data['first_name']) ?>!</h2>

B. In Management Tables

When Staff view passenger names or ID numbers, those values are "sanitized" for the browser.

- **File**: [

view_admin_bookings.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/view_admin_bookings.php)

Name: <?= htmlspecialchars($row['passenger_name']) ?>

C. In Customer Feedback

Since customers can type long comments, this is the most likely place for someone to accidentally (or intentionally) type characters that could mess up the HTML.

- **File**: [

view_feedback.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/view_feedback.php)

<td><?= htmlspecialchars($row['comments']) ?></td>

To understand **how** htmlspecialchars() works, you have to think like a Web Browser.

A browser reads code from top to bottom. If it sees a < character, it thinks: *"Aha! This is the start of a command (a tag) like or <script>."*

htmlspecialchars() works like a "translator" that changes those trigger characters into safe codes called **HTML Entities**.

The Transformation Process

When you pass a string through htmlspecialchars(), it performs a find-and-replace on these 5 key characters:

Real Character	HTML Entity (The Safe Code)	Browser's Behavior
<	< (stands for **L**ess **T**han)	Displays < but doesn't start a tag.
>	> (stands for **G**reater **T**han)	Displays > but doesn't end a tag.
&	&	Displays & without starting an entity.
"	"	Displays " without breaking HTML attributes.
'	'	Displays ' safely.

A Concrete Example

Imagine a malicious user types this into the "Passenger Name" box: Hacker

1. Without htmlspecialchars():

PHP sends exactly that to the browser:

<td>Hacker</td>

Result: The browser sees and makes the word **Hacker** bold. This is harmless for bolding, but dangerous if they typed <script> (which could steal your cookies).

2. With htmlspecialchars():

PHP "translates" the string before sending it:

echo htmlspecialchars("Hacker");

The browser receives this:

<td><b>Hacker</b></td>

Result: The browser doesn't see any "tags". It just sees text starting with <. It displays the literal characters **Hacker** on the screen. **The "code" has been neutralized.**

The technical "How" (Under the hood)

1. **Scanning**: The function scans your string character by character.
2. **Matching**: It checks if the character is in its "Special List" (&, ", ', <, >).
3. **Replacing**: If it finds a match, it swaps the character for the multi-character entity code.
4. **Encoding**: By default, it uses **UTF-8** encoding to ensure it handles international symbols correctly as well.

Fetching dashboard names

To fetch and display user names in the dashboards, the system follows a two-step process: **fetching during login** and **retrieving from the session** (or database) on the dashboard pages.

Here is the breakdown of where this logic is located in your code:

1. Fetching the Name During Login

When a user logs in, their information is retrieved from the users table and stored in a PHP session variable.

- **File:** [

account2.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/account2.php)

- **The Query (Line 117):**

$stmt = $conn->prepare("SELECT user_id, password, role, first_name, last_name FROM users WHERE email = ?");

- **Storing in Session (Line 134):** After verifying the password, the first and last names are combined and stored in $_SESSION['name']:

$_SESSION['name'] = $f_name . " " . $l_name;

---

2. Displaying the Name in Dashboards

Each dashboard then accesses this session data to greet the user.

Admin Dashboard

- **File:** [

admin_dashboard.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/admin_dashboard.php)

- **Display Logic (Line 136):**

Welcome back, <?= htmlspecialchars($_SESSION['name']) ?>. You have full access to manage the system.

Agent Dashboard

- **File:** [

agent_dashboard.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/agent_dashboard.php)

- **Retrieval (Line 18):** $agent_name = $_SESSION['name'];
- **Display Logic (Line 82):**

Welcome back, <?= htmlspecialchars($agent_name) ?>. Manage your daily operations below.

Passenger Dashboard

The passenger dashboard is slightly different; it performs a fresh database query to get just the first name for a more personal greeting.

- **File:** [

dashboard.php](file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/dashboard.php)

- **The Query (Lines 27–30):**

```
$stmt = $conn->prepare("SELECT first_name FROM users WHERE user_id = ?");

$stmt->bind_param("i", $user_id);

$stmt->execute();

$user_data = $stmt->get_result()->fetch_assoc();
```

- **Display Logic (Line 94):**

```
<p>Welcome back, <?= htmlspecialchars($user_data['first_name']) ?>! What would you like to do today?</p>
```

Summary of Key Locations:

Summary of Key Locations:

Action	File	Relevant Lines
Data Fetching	[**account2.php**] (file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/account2.php)	117, 122, 134
Admin Greeting	[**admin_dashboard.php**] (file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/admin_dashboard.php)	136
Agent Greeting	[**agent_dashboard.php**] (file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/agent_dashboard.php)	82
User Greeting	[**dashboard.php**] (file:///c:/Users/Admin/Desktop/UDA/Project/IBBS_PROTOTYPE/pages/dashboard.php)	27–30, 94